

Plano — Assistente do LangNet (um agente que faz, dentro do LangNet, o que o Claude Code faz hoje)

Versão 1.1 · 01/10/2026 · Projeto de teste: BioByte Sentinela v5

O que mudou da 1.0: nova seção 4.6 (como o assistente conhece o LangNet: o manual, os procedimentos por etapa, a memória do projeto e as ferramentas), nova fase **F0.5** e o primeiro rascunho do manual em docs/assistente/manual_langnet_v0.1.md. Também ficou registrado como a sessão é identificada (4.3) e que ela só é acordada quando o usuário manda uma mensagem.

1. O que se quer

Hoje o trabalho de conduzir um projeto pelo pipeline do LangNet é feito por uma sessão do Claude Code no terminal: ela carrega os documentos, dispara cada etapa, espera a geração, lê o resultado, confere no banco e nas telas, pede correções ao agente da etapa, aprova e segue. O objetivo é ter **esse mesmo papel dentro do próprio LangNet**:

- um ícone de chat no canto direito de qualquer tela do projeto abre um painel de conversa;
- quem conversa é o mesmo modelo (Claude Opus, pela assinatura Max, na máquina que já serve o LangNet);
- o assistente **faz** as coisas (gera, refina, compara, aprova, confere), não só responde;
- ele **lembra** de tudo o que já fez no projeto, no mesmo nível de contexto desta sessão;
- tudo fica registrado no projeto, como já acontece com as conversas de cada etapa;
- o modo manual continua existindo: cada etapa pode ser feita à mão ou pelo assistente.

2. O que já existe e foi conferido

Peça	Situação
Ponte OpenAI-compatível do Claude (192.168.1.100:4443, modelo claude-code)	Funciona, sem custo por token. Mas não executa ferramentas (devolve texto mesmo quando recebe tools) e não guarda estado entre chamadas; sem fluxo, corta respostas grandes. Serve para gerar documentos, não para ser o agente.
Hub Sessões (/mnt/data1/progpython/gerenciadorsessoesclaude)	Já dirige sessões do Claude Code com <code>claude -p --resume <sessão></code> : o contexto vive no transcrito em disco, sem terminal aberto. É exatamente o mecanismo de memória desta sessão.
LangNet: conversas por etapa	Tabelas <code>specification_chat_messages</code> , <code>data_model_chat_messages</code> , <code>ui_spec_chat_messages</code> , <code>tool_chat_messages</code> etc. já registram cada pedido e resposta por projeto.
LangNet: API REST	Todas as etapas têm endpoints (gerar, refinar, versões, aprovar) — é o que esta sessão usa hoje, com o token de login.

3. A decisão de arquitetura

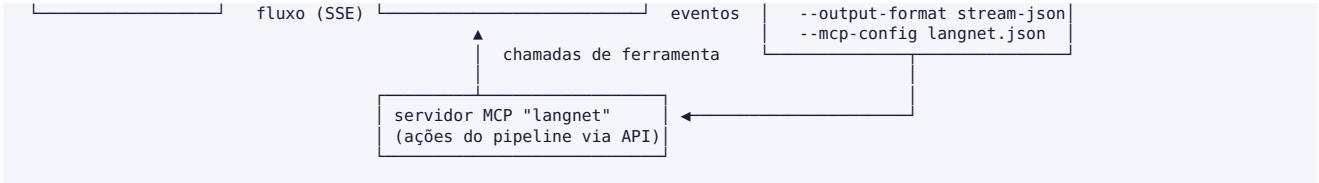
Recomendação: uma sessão do Claude Code por projeto, rodando na máquina do plano Max, acordada pelo LangNet a cada mensagem com `--resume`, e com as ações do LangNet oferecidas a ela como ferramentas por um servidor MCP.

Por que esse caminho e não a ponte 4443:

- **Mesmo contexto que esta sessão.** O `--resume` reabre o transcrito inteiro; quando ele cresce, o próprio Claude Code compacta, como aqui. Não é preciso reinventar memória de conversa.
- **Ferramentas de verdade.** O Claude Code chama ferramentas MCP nativamente; a ponte não chama.
- **Sem custo por token**, pela assinatura Max — a mesma de hoje.
- **Já provado em casa** pelo Hub Sessões.

O caminho alternativo (ponte 4443 + um laço de agente escrito por nós, com ações em JSON no texto e a memória montada pelo LangNet a cada chamada) fica como plano B, caso o headless não possa rodar naquela máquina. Ele exige reimplementar contexto e compactação, e por isso fica pior.





4. As peças a construir

4.1 Serviço de sessões (na máquina Max)

Um serviço pequeno, com token (mesmo esquema da ponte 4443), que: - recebe {projeto, mensagem} e roda `claude -p --resume <sessão_do_projeto> --output-format stream-json` com a configuração MCP do LangNet e a lista de ferramentas permitidas; - cria a sessão na primeira mensagem do projeto e guarda o id; - devolve os eventos em fluxo: texto do modelo, chamada de ferramenta, resultado, fim; - garante **uma execução por projeto de cada vez** (fila), e permite interromper; - tem uma pasta de trabalho por projeto com um `CLAUDE.md` gerado pelo LangNet (ver 4.5).

Pode ser um módulo novo do Hub Sessões, que já tem o daemon e o `--resume`, em vez de um serviço do zero.

4.2 Servidor MCP "langnet" (as mãos do assistente)

Expõe as ações do LangNet como ferramentas, todas por cima da API REST que já existe:

Grupo	Ferramentas
Ler	<code>estado_do_projeto</code> (etapas, versões, aprovações) · <code>ler_documento</code> (etapa, versão, trecho) · <code>comparar_versoes</code> (etapa, v1, v2) · <code>consultar_banco</code> (só leitura) · <code>ler_conversas_da_etapa</code>
Fazer	<code>carregar_documento</code> · <code>gerar_etapa</code> (etapa, origem, versão, instrução) · <code>refinar_etapa</code> (etapa, pedido) · <code>aprovar_versao</code> · <code>rodar_portao</code> · <code>gerar_codigo</code> · <code>implantar</code> · <code>rodar_casos_de_teste</code>
Esperar	<code>acompanhar_tarefa(id)</code> — as gerações levam minutos; a ferramenta devolve o andamento, e o painel mostra o progresso
Ver	<code>capturar_tela(rota)</code> — foto da tela do LangNet ou do app gerado, para o modelo conferir o que o usuário veria

Regras do servidor: - **o programa executa, o modelo decide** (a regra que já vale no LangNet); - aprovar, implantar e apagar **pedem confirmação** no painel antes de rodar; - cada chamada fica registrada com quem pediu, o que entrou e o que saiu.

4.3 Backend do LangNet — router /assistant

- `POST /projects/{id}/assistant/messages` — envia a mensagem e devolve o fluxo de eventos (SSE);
- `GET /projects/{id}/assistant/messages` — histórico do projeto;
- `POST /projects/{id}/assistant/confirm/{acao}` — confirmação de ação sensível;
- `POST /projects/{id}/assistant/stop` — interromper.

Tabelas novas (migração 032): - `assistant_sessions` — projeto, **id da sessão do Claude** (o identificador que o `--resume` usa), estado, criado/atualizado. Uma sessão ativa por projeto; um botão "nova conversa" abre outra e leva a memória do projeto junto; - `assistant_messages` — papel, texto, eventos de ferramenta, versões criadas ou aprovadas, data; - `assistant_project_memory` — fatos duráveis do projeto (decisões, regras, pendências), editáveis.

4.4 Frontend — o painel

- ícone de chat no canto direito do cabeçalho, em todas as telas dentro de um projeto;
- painel lateral (gaveta) com a conversa, a resposta chegando em fluxo e cada passo do agente visível ("gerando a especificação a partir dos requisitos v1...", "conferindo a tabela no banco...");
- botões de confirmar e cancelar quando o assistente pede para aprovar, implantar ou apagar;
- links diretos para a versão criada, a comparação e a tela afetada;
- o assistente sabe em que tela o usuário está e usa isso como contexto da mensagem.

4.5 Quando a sessão é acordada

Só quando o usuário manda uma mensagem. Abrir o projeto ou o painel apenas mostra o histórico, lido do banco do LangNet — não acorda o Claude nem gasta a cota do plano Max. Cada mensagem roda `claude -p --resume <id>`: o Claude reabre a conversa, faz o trabalho, responde e encerra.

4.5.1 Memória e contexto — "no mesmo nível desta sessão"

Três camadas, como aqui: 1. **Transcrito da sessão** (-- resume): a conversa inteira, compactada automaticamente quando cresce; 2. **Memória do projeto** (assistant_project_memory → gerada como CLAUDE.md da pasta do projeto): o que precisa sobreviver a qualquer compactação — regras do pipeline (origem + versão → gerar → refinar → aprovar), decisões tomadas, pendências, lições (ex.: "não regerar para descobrir"); 3. **Estado real lido do banco** a cada pedido, pelas ferramentas: o assistente não confia na lembrança para saber em que versão cada etapa está — ele consulta.

4.6 Como o assistente conhece o LangNet (o "contexto inicial")

Não é preciso RAG. O conhecimento cabe inteiro no contexto (dezenas de milhares de tokens num modelo de um milhão) e entra em quatro camadas — as mesmas que fazem esta sessão do Claude Code conhecer o LangNet:

Camada	O que é	Onde fica	Quando entra
Manual do LangNet	etapas, ordem, o que entra e sai de cada uma, o ritual de toda etapa, o que um bom documento de cada etapa tem de ter, as regras de conduta	docs/assistente/manual_langnet.md, versionado; vira o CLAUDE.md da pasta de cada projeto	em toda sessão, desde a primeira mensagem
Procedimentos por etapa (skills do Claude Code)	um roteiro por etapa: como gerar, como revisar, o que conferir, que correção pedir	docs/assistente/skills/<etapa>/SKILL.md	só quando o assunto aparece ("revise a especificação" carrega a da especificação)
Memória do projeto	nome, domínio, documentos, versão de cada etapa, decisões, pendências	tabela assistant_project_memory, gerada do banco	em toda mensagem; sobrevive a qualquer resumo da conversa
Ferramentas	cada ferramenta descreve o que faz e o que devolve; o estado real vem sempre do banco	servidor MCP "langnet"	quando o assistente age

O documento do cliente (a ata, no BioByte) não fica no contexto: é lido pela ferramenta quando preciso. RAG só se justificaria para buscar em muitos documentos grandes de uma vez — e entraria como mais uma ferramenta, não como a base do conhecimento.

Exemplo — "abra a última versão do documento de requisitos e comece a especificação": o manual diz que a especificação parte dos requisitos → estado_do_projeto diz qual é a última versão → entra o procedimento da etapa de Especificação → o assistente escreve a instrução, chama gerar_etapa, acompanha até voltar, lê o documento, confere pela lista do que ele tem de ter, e responde com o que achou e a proposta de correção.

5. Fases

Fase	Entrega	Como se prova
F0	Prova de conceito: claude -p -- resume na máquina Max com um servidor MCP de uma ferramenta (estado_do_projeto)	duas mensagens seguidas; a segunda lembra da primeira; a ferramenta devolve o estado real do BioByte
F0.5	Manual do LangNet e procedimentos por etapa (rascunho já em docs/assistente/manual_langnet_v0.1.md)	o manual revisado pelo autor; um procedimento por etapa; numa sessão de teste, o assistente explica corretamente o ritual e o que conferir em três etapas escolhidas
F1	Serviço de sessões + router /assistant + migração 032	conversa pela API, registrada no banco, com fluxo
F2	Ferramentas de leitura (ler, comparar, consultar banco, conversas da etapa)	o assistente responde corretamente "em que versão está a especificação e o que mudou da v1 para a v2"
F3	Painel de chat no React	conversa pela tela, com os passos visíveis
F4	Ferramentas de ação + confirmação + acompanhamento de tarefas longas	o assistente refina o modelo de dados do BioByte por conversa e a versão nova aparece na tela da etapa
F5	Memória do projeto e retomada	fechar o navegador, voltar no dia seguinte: o assistente sabe onde parou
F6	Captura de telas e bateria no app gerado	

Fase	Entrega	Como se prova
		o assistente confere a tela da tarefa e o resultado no banco, como esta sessão faz
F7	Aceitação	refazer pelo assistente uma etapa do BioByte já feita aqui e comparar o resultado

6. Riscos e o que conferir antes

- **A máquina Max precisa ter o Claude Code instalado e logado** na assinatura (a ponte 4443 indica que sim, mas é a primeira coisa da F0).
- **Cotas do plano Max:** cada mensagem reabre o transcrito; sessões longas consomem mais. Limitar uma execução por projeto e acompanhar o uso.
- **Permissões:** a sessão do assistente roda **só com as ferramentas MCP do LangNet**, sem terminal livre na máquina do LangNet nem acesso de escrita ao banco fora das ações previstas.
- **Rede:** a máquina Max precisa alcançar o servidor MCP e a API do LangNet (rede local).
- **Gerações longas:** a mensagem não pode ficar presa minutos esperando; a ação devolve um id e o acompanhamento informa o andamento.
- **Falha não pode passar calada:** erro de ferramenta aparece como erro no painel, nunca como sucesso.

7. Próximo passo

Revisar o rascunho do manual (F0.5) e executar a **F0** (prova de conceito na máquina Max) — as duas podem andar juntas. Se o - - resume com MCP funcionar lá, o resto é construção; se não funcionar, cai-se no plano B (ponte 4443 + laço próprio) antes de construir qualquer tela.